

Отчёт по проекту

Клиент-серверное приложение для графического
отображения ветвящейся функции

Дисциплина: Технологии и методы программирования

Язык: C++17 | Фреймворк: Qt 6 | БД: SQLite

Введение

- 1.1. Цель и задачи проекта
- 1.2. Применяемые технологии и инструменты
- 1.3. Функциональные возможности приложения
- 1.4. Структура отчета

- **Обзор применяемых технологий**

- 2.1. Docker
- 2.2. Doxygen
- 2.3. Qt Creator
- 2.4. Qt Test
- 2.5. GitHub

- **Архитектура клиент-серверного приложения**

- 3.1. Общая структура
- 3.2. Клиентская часть
- 3.3. Серверная часть
- 3.4. Протокол взаимодействия

- **Графический модуль: построение графиков**

- 4.1. Принцип работы
- 4.2. Алгоритм построения
- 4.3. Интерфейс управления (ползунки)

- **Тестирование**

- **Документирование**
- **Двухфакторная аутентификация и управление доступом**
 - 7.1. Регистрация нового пользователя
 - 7.2. Авторизация с двухфакторной проверкой
 - 7.3. Восстановление пароля
- **Серверное хранение данных и управление состояниями**
 - 8.1. Локальное кэширование на клиенте (SQLite)
 - 8.2. Постоянное хранение на сервере (PostgreSQL)
 - 8.3. Управление временными состояниями (Кэш кодов и сессий)
- **Заключение**

1. Введение

Данное клиент-серверное приложение разработано в рамках дисциплины «Технологии и методы программирования». Основная цель проекта --- графическое отображение ветвящейся (кусочной) функции, которая задаётся тремя параметрами, управляемыми интерактивными ползунками. Результат работы программы --- три графика, построенные в одной системе координат на основе единой формулы с различными значениями параметров.

Проект реализован на языке C++17 с использованием фреймворка Qt 6. Клиентская часть предоставляет графический интерфейс (GUI), серверная часть обеспечивает обработку данных, хранение в базе SQLite и отправку уведомлений по email через SMTP. Весь серверный стек контейнеризирован с помощью Docker, что гарантирует воспроизводимость среды и удобство развёртывания.

В настоящем отчёте подробно рассматриваются все технологии, применённые в проекте: Docker для контейнеризации, Doxygen для генерации документации, Qt Creator как среда разработки, Qt Test для модульного тестирования и GitHub для управления версиями. Отдельное внимание уделяется архитектуре клиент-серверного взаимодействия и алгоритму построения графиков.

2. Обзор применяемых технологий

Проект использует набор современных инструментов и технологий, каждый из которых выполняет свою роль в жизненном цикле разработки. Ниже приведён подробный обзор каждой из них.

2.1. Docker

Docker --- это платформа для контейнеризации приложений, позволяющая упаковать программу вместе со всеми её зависимостями в единый переносимый контейнер. В данном проекте Docker используется для развёртывания серверной части приложения.

Основные преимущества Docker в контексте данного проекта:

Изоляция. Серверная часть работает в изолированном контейнере на базе Ubuntu 22.04, что исключает конфликты с системными библиотеками хост-машины. Все зависимости --- библиотеки Qt 6, драйвер SQLite, утилиты для работы с сетью --- находятся внутри контейнера и не влияют на хост.

Портативность. Контейнер можно запустить на любой операционной системе, где установлен Docker: Linux, macOS, Windows. Это критически важно для командной разработки и демонстрации проекта --- достаточно одной команды для запуска.

Воспроизводимость. Файл Dockerfile содержит точную инструкцию сборки среды. Любой разработчик может воссоздать идентичное окружение, что исключает проблему «работает у меня».

Масштабируемость. В случае необходимости сервер можно легко масштабировать, запустив несколько экземпляров контейнера или используя Docker Compose для оркестрации.

В проекте Docker-контейнер экспортирует TCP-порт 33333 для приёма клиентских подключений. Сборка контейнера выполняется через скрипты, размещённые в директории docker/. Серверный контейнер включает в себя: исполняемый файл сервера, библиотеки Qt 6 (Network, SQL), SQLite-драйвер и конфигурационные файлы.

2.2. Doxygen

Doxygen --- это стандартный инструмент для автоматической генерации документации из исходного кода. Он анализирует специальным образом оформленные комментарии в коде и создаёт структурированную документацию в различных форматах: HTML, LaTeX (PDF), RTF.

В данном проекте Doxygen используется для генерации HTML-документации на русском языке. Конфигурационный файл Doxyfile размещён в директории docs/ и содержит настройки проекта:

--- INPUT --- путь к исходным файлам (директории UI/Timpproject/ и server/);
--- OUTPUT_DIRECTORY --- путь для сохранения сгенерированной документации (docs/html/); --- PROJECT_NAME --- название проекта; ---
GENERATE_HTML, GENERATE_LATEX --- форматы вывода; ---
SOURCE_BROWSER --- включение browsable-списка исходников; ---
EXTRACT_ALL --- извлечение документации из всех элементов, включая те, что не имеют комментариев.

Для каждого класса, метода и структуры данных в проекте написаны Doxygen-комментарии формата `/** ... */`, содержащие описание назначения, входные и выходные параметры, а также информацию о возвращаемых значениях. Генерация документации выполняется командой «doxygen Doxyfile» из директории docs/.

2.3. Qt Creator

Qt Creator --- кросс-платформенная интегрированная среда разработки (IDE), специально оптимизированная для работы с фреймворком Qt. Она предоставляет полный набор инструментов для разработки на C++: редактор кода с подсветкой синтаксиса, автодополнение, отладчик, профилировщик и визуальный редактор форм (Qt Designer).

В проекте Qt Creator используется как основная IDE. Основные возможности, задействованные при разработке:

Визуальный редактор интерфейсов. Qt Designer позволяет создавать графический интерфейс клиента в режиме «перетаски и настрой». Окно

главного класса QMainWindow содержит область отображения графиков (наследник QWidget) и панель управления с тремя ползунками (QSlider).

Менеджер проектов. Qt Creator поддерживает систему сборки qmake/CMake. Проект организован в несколько целей (targets): GUI-клиент (библиотека или приложение), сервер (приложение), тесты (Qt Test).

Отладчик. Встроенная интеграция с GDB/LLDB позволяет отлаживать как клиентскую, так и серверную часть, устанавливать точки останова, просматривать стек вызовов и значения переменных в реальном времени.

Профилировщик. Встроенный инструмент для анализа производительности помогает выявить узкие места при отрисовке графиков и обработке сетевых запросов.

Сетевой монитор. Qt Creator предоставляет инструмент для мониторинга TCP/UDP-соединений, что упрощает отладку клиент-серверного протокола.

2.4. Qt Test

Qt Test (QTestLib) --- это фреймворк для модульного тестирования, входящий в состав Qt. Он предоставляет макросы и вспомогательные функции для написания тестов на C++, проверки условий и сравнения значений.

Основные особенности Qt Test, использованные в проекте:

Макросы проверки. QCOMPARE(actual, expected) --- сравнение значений с выводом детального сообщения об ошибке. QVERIFY(condition) --- проверка произвольного условия. QASSERT --- аналог с немедленным прерыванием теста.

Data-driven тесты. Qt Test поддерживает параметризованные тесты, когда один и тот же набор проверок выполняется с различными входными данными. Это особенно полезно для тестирования вычислительного ядра --- проверка корректности формулы для различных значений параметров.

Тестовые слоты. Каждый тестовый метод объявляется как слот класса-наследника QObject. Фреймворк автоматически обнаруживает и запускает все тестовые слоты.

Интеграция с Qt Creator. Тесты встроены в проект и могут быть запущены непосредственно из IDE с автоматическим отображением результатов (пройденные/не пройденные тесты, время выполнения).

В проекте протестированы: корректность вычисления значений кусочной функции, работа с граничными значениями параметров, корректность разбиения области определения на интервалы, а также обработка ошибок при некорректных входных данных.

2.5. GitHub

GitHub используется как хостинг для хранения исходного кода и управления версиями проекта. Основные аспекты использования GitHub в данном проекте:

Ветвление. Основная ветка `main` содержит стабильную версию кода. Функциональные изменения разрабатываются в отдельных ветках (`feature branches`) и сливаются в `main` через `Pull Requests`.

`Pull Requests`. Каждое изменение проходит через процесс код-ревью: разработчик создаёт PR, описывает внесённые изменения, и после проверки код сливается в основную ветку. Это обеспечивает контроль качества и документирование всех изменений.

Issues. Для отслеживания задач, ошибок и предложений по улучшению используются GitHub Issues. Каждая задача имеет метки (`labels`), приоритет и статус.

Actions (CI/CD). Настроены автоматические workflow для сборки проекта, запуска тестов и генерации документации при каждом коммите. Это гарантирует, что код всегда в рабочем состоянии.

README.md. В корне репозитория размещён файл с описанием проекта, инструкциями по сборке, структурой директорий и используемыми технологиями.

3. Архитектура клиент-серверного приложения

3.1. Общая структура

Приложение построено по классической клиент-серверной архитектуре. Клиент и сервер --- самостоятельные исполняемые файлы, взаимодействующие по сети через TCP-протокол. Сервер размещён в Docker-контейнере, клиент запускается локально.

Структура проекта: --- UI/Timpproject/ --- исходный код GUI-клиента (Qt Widgets); --- server/ --- исходный код TCP-сервера (Qt Network + Qt SQL); --- docker/ --- Dockerfile и скрипты для сборки/запуска контейнера; --- tests/ --- модульные тесты (Qt Test); --- docs/ --- конфигурация Doxygen и сгенерированная документация.

3.2. Клиентская часть

Клиент реализован как десктопное приложение на Qt 6 Widgets. Его основные компоненты:

Главное окно (MainWindow). Содержит виджет для отрисовки графиков и панель управления. Панель управления оснащена тремя горизонтальными ползунками (QSlider), каждый из которых отвечает за один параметр кусочной функции. Под каждым ползунком отображается текущее числовое значение параметра.

Графический виджет (PlotWidget). Наследник QWidget, выполняющий ручную отрисовку QPainter. Отображает три графика (кривые) в одной системе координат. Каждая кривая имеет свой цвет и отображается в соответствии с легендой.

Сетевой модуль. Использует QTcpSocket для подключения к серверу. При изменении параметра (перемещении ползунка) клиент отправляет запрос серверу и получает вычисленные значения для отрисовки.

Почтовый модуль. Реализован через QSslSocket (SMTP, SSL, порт 465). Позволяет отправлять результаты (изображение графика или числовые данные) по email.

3.3. Серверная часть

Сервер --- это консольное приложение на Qt 6 (модули Network и SQL). Оно выполняет следующие функции:

Приём и обработка TCP-запросов. Сервер слушает порт 33333 и принимает подключения от клиентов. Текстовый протокол использует разделитель || для полей сообщения.

Вычисления. При получении параметров (значений трёх ползунков) сервер вычисляет значения кусочной функции на заданном интервале и возвращает клиенту набор точек для построения трёх графиков.

Хранение данных. Результаты вычислений сохраняются в базе SQLite через QSqlDatabase (драйвер QSQLITE). Это позволяет накапливать историю вычислений и при необходимости восстанавливать предыдущие результаты.

SMTP-интеграция. Сервер может отправлять email-уведомления с результатами вычислений через Gmail SMTP (порт 465, SSL).

3.4. Протокол взаимодействия

Клиент и сервер обмениваются текстовыми сообщениями через TCP на порту 33333. Протокол прост и использует разделитель || между полями сообщения.

Пример запроса клиента:

REQUEST||param1||param2||param3||interval_start||interval_end||num_points

Пример ответа сервера:

RESPONSE||graph1_points||graph2_points||graph3_points

Каждый набор точек представлен в формате «x1:y1;x2:y2;...» где x --- абсцисса, y --- ордината. Протокол выбран максимально простым для обеспечения совместимости и возможности отладки вручную.

4. Графический модуль: построение графиков

4.1. Принцип работы

Основная функциональность приложения --- построение трёх графиков ветвящейся (кусочной) функции в одной системе координат. Функция задаётся формулой, которая принимает три параметра: a , b и c . Каждый из трёх графиков соответствует различным комбинациям или вариантам формулы с одним и тем же набором параметров, управляемых ползунками.

Ветвящаяся (кусочная) функция --- это функция, заданная различными аналитическими выражениями на различных интервалах области определения. Формула может содержать условия, определяющие, какое выражение используется для данного значения x .

Ползунки позволяют интерактивно изменять параметры a , b , c в пределах заданных диапазонов. При каждом перемещении ползунок происходит пересчёт значений функции и перерисовка всех трёх графиков. Это обеспечивает визуальную интерактивность и позволяет исследовать поведение функции при различных параметрах.

4.2. Алгоритм построения

Алгоритм построения графиков реализован следующим образом:

- Определение области отображения. По умолчанию используется диапазон $x \in [x_{\min}, x_{\max}]$, который разбивается на N равных интервалов (количество точек задаётся параметром `num_points`).
- Вычисление значений. Для каждой точки x_i вычисляются три значения: $y1 = f1(x_i, a, b, c)$, $y2 = f2(x_i, a, b, c)$, $y3 = f3(x_i, a, b, c)$. Выбор конкретного выражения внутри кусочной функции определяется условиями ветвления.
- Масштабирование. Вычисленные значения y нормализуются к размерам виджета с учётом соотношения масштабов осей X и Y (`aspect ratio`).

- Отрисовка. Используя QPainter, точки соединяются линиями, формируя непрерывную кривую. Каждая из трёх кривых отрисовывается своим цветом. Добавляются оси координат, сетка, подписи делений и легенда.

4.3. Интерфейс управления (ползунки)

Интерфейс управления представлен тремя горизонтальными ползунками QSlider. Каждый ползунок отвечает за один параметр кусочной функции:

Ползунок 1 (параметр a). Диапазон значений: $[a_min, a_max]$ с шагом $step_a$. Влияет на амплитуду или наклон функции в зависимости от конкретной формулы.

Ползунок 2 (параметр b). Диапазон значений: $[b_min, b_max]$ с шагом $step_b$. Определяет смещение или масштаб отдельных ветвей функции.

Ползунок 3 (параметр c). Диапазон значений: $[c_min, c_max]$ с шагом $step_c$. Управляет пороговыми значениями условий ветвления или другими параметрами формы.

Под каждым ползунком размещена текстовая метка (QLabel), отображающая текущее числовое значение параметра. При перемещении ползунка значение обновляется в реальном времени, и график перерисовывается немедленно (событие `valueChanged` слота QSlider).

Такая реализация обеспечивает плавное и отзывчивое управление: пользователь видит мгновенную реакцию графика на изменение каждого параметра, что делает исследование поведения функции интуитивным и наглядным.

5. Тестирование

Тестирование проекта выполнено с помощью фреймворка Qt Test. Набор тестов размещён в директории `tests/` и организован в виде нескольких тестовых классов:

Тесты вычислительного ядра. Проверяется корректность вычисления значений кусочной функции для различных комбинаций параметров, включая граничные значения и точки разрыва ветвей.

Тесты клиента. Верификация корректности формирования запросов, парсинга ответов сервера, а также обработки сетевых ошибок (разрыв соединения, таймаут).

Тесты сервера. Проверка обработки входящих соединений, корректности работы с SQLite, а также SMTP-отправки.

Результаты тестов автоматически выводятся в консоль и могут быть просмотрены в Qt Creator. Каждый тест содержит описание проверяемого сценария, ожидаемый и фактический результат.

6. Документирование

Документация проекта генерируется автоматически с помощью Doxygen. Конфигурационный файл Doxyfile находится в директории docs/ и содержит все необходимые настройки для генерации HTML-документации.

Для каждого компонента проекта (классы, методы, структуры данных) написаны Doxygen-комментарии, описывающие назначение, параметры и возвращаемые значения. Документация генерируется на русском языке.

Сгенерированная HTML-документация размещена в docs/html/ и включает:

--- индекс классов и файлов; --- подробные описания каждого класса и метода; --- граф зависимостей (GraphViz, если установлен); --- browsable-список исходных файлов с подсветкой синтаксиса.

Генерация выполняется командой «doxygen Doxyfile» из директории docs/. Результатом является набор HTML-страниц, доступных для просмотра в любом веб-браузере.

7. Двухфакторная аутентификация и управление доступом

В рамках проекта реализована многоуровневая система аутентификации и восстановления доступа, повышающая безопасность приложения. Этот механизм охватывает как процесс регистрации новых пользователей, так и вход существующих, а также процедуру сброса забытого пароля. Ключевой особенностью является двухфакторная верификация через подтверждение по электронной почте (2FA).

7.1. Регистрация нового пользователя

Процесс регистрации разделён на два логических этапа для предотвращения создания невалидных или мошеннических учётных записей.

На первом этапе пользователь заполняет форму регистрации, указывая желаемый логин (мин. 3 символа), email-адрес и пароль. Пароль проходит строгую валидацию на клиентской стороне: он должен содержать не менее 8 символов и состоять только из латинских букв и цифр. После проверки корректности введённых данных, клиент отправляет серверу запрос `reg_request_code&` с указанным email-адресом. Сервер, в свою очередь, генерирует случайный шестизначный код, сохраняет его в оперативной памяти с отметкой времени (время жизни кода — 5 минут) и отправляет этот код на указанный email через настроенный SMTP-сервер.

Второй этап заключается в подтверждении. Пользователь вводит полученный по почте код в специальное поле интерфейса. Клиент отправляет серверу запрос `reg_confirm&` с логином, паролем, email-адресом и введённым кодом. Сервер проверяет совпадение кода и его актуальность (не истёк ли таймаут). При успешной проверке данные о новом пользователе (логин, хеш пароля, email) сохраняются в базу данных, а временный код удаляется. После завершения регистрации сервер отправляет приветственное письмо, а клиент автоматически переходит к окну авторизации.

7.2. Авторизация с двухфакторной проверкой

Процесс входа в систему также является двухэтапным и требует от пользователя не только знания пароля, но и доступа к его почтовому ящику.

Первый этап: Пользователь вводит логин и пароль в окне авторизации (LoginTimp). Клиент отправляет серверу запрос `auth_request_code&`. Сервер проверяет наличие пользователя в базе данных и соответствие введённого пароля сохранённому хешу (с использованием соли для повышения криптостойкости). Если данные верны, сервер генерирует новый код подтверждения, привязывает его к логину пользователя и отправляет на email, ассоциированный с этой учётной записью.

Второй этап: После успешной отправки кода интерфейс клиента переключается в режим ожидания кода. Пользователь вводит полученный

шестизначный код. Клиент отправляет запрос `auth_confirm&`. Сервер проверяет код и, если он корректен, генерирует уникальный `sessionToken` (хеш-строка длиной 32 символа), который сохраняется в серверном кэше сессий и отправляется обратно клиенту как подтверждение успешного входа.

Блокировка аккаунта: Для предотвращения атак перебора паролей на серверной стороне реализован механизм автоматической блокировки учётной записи. После трёх неудачных попыток ввода пароля учётная запись временно блокируется на 30 секунд. Информация о количестве неудачных попыток и времени блокировки хранится в базе данных.

7.3. Восстановление пароля

Процедура сброса пароля (`ForgotPassword`) реализована по схожему с регистрацией принципу. Пользователь вводит свой логин, после чего сервер находит связанный с ним email-адрес, генерирует код восстановления и отправляет его на почту. После ввода верного кода пользователю предоставляется возможность задать новый пароль, который затем обновляется в базе данных. На всех этапах, связанных с отправкой кодов, используется централизованный модуль `EmailSender`, что обеспечивает единообразие и упрощает поддержку логики уведомлений.

8. Серверное хранение данных и управление состояниями

Надёжное и безопасное хранение данных является критически важным аспектом для любого серверного приложения. В разработанной системе реализован гибридный подход к управлению данными, сочетающий использование двух различных систем управления базами данных (СУБД) для решения разных задач.

8.1. Локальное кэширование на клиенте (SQLite)

Для повышения автономности и отзывчивости клиентского приложения на стороне клиента используется встраиваемая СУБД `SQLite`. Основной файл базы данных (`users.db`) создаётся в директории с исполняемым файлом клиента. Эта база данных служит для локального кэширования информации о пользователе (например, логин, статус последнего входа) и уменьшения

количества сетевых запросов к серверу для получения часто используемых, но редко изменяющихся данных. Класс DatabaseManager (клиентская часть) предоставляет потокобезопасный интерфейс Singleton для работы с этой базой: инициализация, создание таблиц, операции чтения/записи.

8.2. Постоянное хранение на сервере (PostgreSQL)

Основным хранилищем пользовательских данных и критической информации является серверная база данных на основе PostgreSQL. Выбор PostgreSQL обусловлен его высокой надёжностью, поддержкой сложных запросов и конкурентного доступа, что необходимо для многопользовательской работы в будущем. Модуль ServerDB инкапсулирует всю логику взаимодействия с PostgreSQL через библиотеку libpq.

Структура основной таблицы users включает следующие поля:

- id: Уникальный автоинкрементный идентификатор.
- login: Уникальное имя пользователя (используется для входа).
- email: Уникальный адрес электронной почты (используется для верификации).
- password_hash: Хеш пароля, вычисленный с использованием соли (алгоритм SHA-256). Хранение только хеша гарантирует, что даже при компрометации базы данных злоумышленник не получит реальные пароли.
- created_at, last_login: Временные метки, используемые для аудита и статистики.
- failed_attempts, lock_until: Поля для реализации механизма блокировки учётной записи (см. раздел 7.2).

8.3. Управление временными состояниями (Кэш кодов и сессий)

Для поддержки двухфакторной аутентификации и сессий серверное приложение использует не только постоянное хранилище, но и кэш в

оперативной памяти. Временные данные, такие как коды подтверждения (`m_tempRegs`, `m_tempAuths`, `m_tempResets`) и активные токены сессий (`m_sessionCache`), хранятся в `QMap` внутри класса `ServerDB`.

- **Коды подтверждения:** Для каждого запроса кода генерируется запись, содержащая сам код, связанный с ним `email` (или `login`) и временную метку создания. Регулярный вызов метода `cleanupExpiredCodes()` при каждой операции обеспечивает автоматическое удаление просроченных (старше 5 минут) кодов, освобождая память и повышая безопасность.
- **Сессионные токены:** Успешная авторизация завершается генерацией токена (`sessionToken`), который помещается в кэш сессий `m_sessionCache`. Это позволяет серверу быстро проверять валидность текущей сессии клиента без выполнения тяжёлых операций записи на диск для каждого запроса. Токен генерируется как хеш от комбинации `login`, временной метки и случайного числа, что делает его уникальным и трудно предсказуемым.

Такой подход к управлению данными позволяет эффективно разделить ответственность между клиентом и сервером, обеспечивая как высокую производительность за счёт локального кэша, так и надёжность и безопасность хранения критически важной информации за счёт использования промышленной СУБД и продуманной политики управления временными состояниями.

9. Заключение

В результате выполнения проекта разработано клиент-серверное приложение для графического отображения ветвящейся функции. Приложение демонстрирует применение широкого спектра современных технологий и инструментов:

- **Docker** обеспечил изоляцию и портативность серверного окружения, что гарантирует воспроизводимость среды и упрощает развёртывание;
- **Doxygen** позволил автоматически сгенерировать структурированную документацию из исходного кода, что облегчает сопровождение проекта;
- **Qt Creator** предоставил полноценную среду разработки с визуальным редактором интерфейсов и отладчиком, ускорив процесс создания приложения;

--- **Qt Test** обеспечил систематическое тестирование всех компонентов системы, повысив надёжность и качество кода;

--- **GitHub** организовал совместную работу и контроль версий, позволив эффективно управлять изменениями в процессе разработки.

Графический модуль позволяет интерактивно исследовать поведение кусочной функции с помощью трёх ползунков, отображая три графика в одной системе координат. Такая реализация обеспечивает наглядность и удобство анализа результатов.

Особое внимание в проекте уделено вопросам безопасности. Реализованная двухфакторная аутентификация существенно повышает защищённость пользовательских учётных записей от несанкционированного доступа. Механизм временной блокировки при неудачных попытках входа предотвращает атаки перебора паролей. Хранение паролей только в виде хеша с солью гарантирует, что даже при компрометации базы данных злоумышленник не получит доступ к паролям пользователей в открытом виде.

Разработанное решение имеет хороший потенциал для дальнейшего расширения. Возможные направления развития проекта включают:

- добавление новых типов функций для визуализации;
- увеличение количества управляемых параметров;
- интеграция с веб-интерфейсом для доступа через браузер;
- расширение серверной части для поддержки множества одновременных клиентов;
- внедрение дополнительных метрик и статистики использования;
- реализация экспорта графиков в различные форматы (PDF, SVG).

Проект полностью соответствует поставленной задаче и демонстрирует успешное применение современных технологий программирования для решения практических задач визуализации данных.